



# Implementing LocationData and LocationProvider

```
location_data.dart > ...  
class LocationData {  
  LocationData({  
    required this.name,  
    required this.latitude,  
    required this.longitude,  
  });  
  
  // an alternative would be a c-tor with positional parameters:  
  // LocationData.pos(this.name, this.latitude, this.longitude);  
  
  final String name;  
  final double latitude;  
  final double longitude;  
}
```

```
lib > location_provider.dart > ...  
1 import 'location_data.dart';  
2  
3 class LocationProvider {  
4   final List<LocationData> _locations = [  
5     LocationData(name: "Aschaffenburg", latitude: 49.97704, longitude: 9.15214),  
6     LocationData(name: "Sydney", latitude: -33.86785, longitude: 151.20732),  
7     LocationData(name: "New York", latitude: 40.71427, longitude: -74.00597),  
8     LocationData(name: "Tokio", latitude: 35.6895, longitude: 139.69171),  
9   ];  
10  
11   String _selectedLocationName = "Aschaffenburg";  
12  
13   bool selectLocation(String name) {  
14     for (var location in _locations) {  
15       if (location.name == name) {  
16         _selectedLocationName = location.name;  
17         return true;  
18       }  
19     }  
20     print("selectLocation: location $name not found");  
21     return false;  
22   }  
23 }
```



## More methods of LocationProvider

```
LocationData get selectedLocation {
  LocationData? result;
  // for (var location in _locations) {
  //   if (location.name == _selectedLocationName) {
  //     result = location;
  //   }
  // }
  // if (result == null) {
  //   throw ("getter selectedLocation: "
  //     "did not find $_selectedLocationName");
  // }

  result = _locations.firstWhere(
    (element) => element.name == _selectedLocationName,
  );

  return result;
}
```

```
List<String> getLocationNames() {
  // List<String> result = [];
  // for (var location in _locations) {
  //   result.add(location.name);
  // }
  // return result;

  return _locations.map((data) => data.name).toList();
}
```

```
LocationData firstWhere(bool Function(LocationData) test, {LocationData Function()? orElse})
Type: LocationData Function(bool Function(LocationData), {LocationData Function()? orElse})
Declared in Iterable in dart:core.
```

The first element that satisfies the given predicate [test].

Iterates through elements and returns the first to satisfy [test].

Example:

```
final numbers = <int>[1, 2, 3, 5, 6, 7];
var result = numbers.firstWhere((element) => element < 5); // 1
result = numbers.firstWhere((element) => element > 5); // 6
result =
  numbers.firstWhere((element) => element > 10, orElse: () => -1); // -1
```

If no element satisfies [test], the result of invoking the [orElse] function is returned. If [orElse] is omitted, it defaults to throwing a [StateError]. Stops iterating on the first matching element.

```
Iterable<T> map<T>(T Function(LocationData) toElement)
Type: Iterable<String> Function(String Function(LocationData))
Declared in Iterable in dart:core.
```

The current elements of this iterable modified by [toElement].

Returns a new lazy [Iterable] with elements that are created by calling `toElement` on each element of this `Iterable` in iteration order.



## Using LocationProvider via “Dependency Injection”

```
forecast_provider.dart > ...
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:lat_lng_to_timezone/lat_lng_to_timezone.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;
import 'location_provider.dart';
import 'weather_data.dart';
```

```
class ForecastProvider {
  ForecastProvider(this._locationProvider);

  final LocationProvider _locationProvider;
  List<WeatherData> hourlyForecast = [];
  bool _initialized = false;
```

```
Future<bool> fetchHourlyForecast() async {
  if (!_initialized) {
    _initialize();
    _initialized = true;
  }

  hourlyForecast.clear();
  var lat = _locationProvider.selectedLocation.latitude;
  var lon = _locationProvider.selectedLocation.longitude;
```

```
void main(List<String> arguments) async {
  var locationProvider = LocationProvider();
  locationProvider.selectLocation("Sydney");
  var forecastProvider = ForecastProvider(locationProvider);

  bool succeeded = await forecastProvider.fetchHourlyForecast();
  if (!succeeded) {
    print("fetchHourlyForecast did not succeed.");
    return;
  }

  print("this is the forecast for ${locationProvider.selectedLocation.name}:");
  for (var e in forecastProvider.hourlyForecast) {
    print("${e.localTime}: ${e.temp}°C, ${e.precipAmount}mm, ${e.cloudCover}%");
  }
}
```

```
Connected to the VM Service.
this is the forecast for Sydney:
2026-03-24 08:00:00.000+1100: 18.1°C, 0.0mm, 69%
2026-03-24 09:00:00.000+1100: 20.6°C, 0.0mm, 46%
2026-03-24 10:00:00.000+1100: 22.5°C, 0.0mm, 55%
```



# Adding unit tests for LocationProvider



The screenshot shows an IDE with a dark theme. On the left, the 'TESTING' sidebar is open, displaying a list of tests. The top section shows '2/2' tests passed in '1.9s'. Below, under the 'weather\_cli\_2016' package, the 'test\location\_provider\_test.dart' file is expanded, showing two tests: 'test selectLocation' (19ms) and 'test getLocationNames' (2.0ms), both marked as passed. The main editor displays the code for 'location\_provider\_test.dart'. It starts with imports for 'package:test/test.dart' and 'package:weather\_cli\_2016/location\_provider.dart'. A 'void main()' function contains two test blocks. The first test, 'test selectLocation', initializes a 'LocationProvider', checks the default location is 'Aschaffenburg', selects 'Sydney', and verifies the result and the provider's state. The second test, 'test getLocationNames', initializes the provider and checks that the 'getLocationNames()' method returns a list of 4 names. The code is as follows:

```
test > location_provider_test.dart > main > test("test getLocationNames")
1  import 'package:test/test.dart';
2  import 'package:weather_cli_2016/location_provider.dart';
3
4  void main() {
5    test('test selectLocation', () {
6      var locationProvider = LocationProvider();
7      // default is Aschaffenburg
8      var location = locationProvider.selectedLocation;
9      expect(location.name, "Aschaffenburg");
10
11     var result = locationProvider.selectLocation("Sydney");
12     expect(result, true);
13     expect(locationProvider.selectedLocation.name, "Sydney");
14
15     result = locationProvider.selectLocation("xxx");
16     expect(result, false);
17     expect(locationProvider.selectedLocation.name, "Sydney");
18   });
19
20   test('test getLocationNames', () {
21     var locationProvider = LocationProvider();
22     var names = locationProvider.getLocationNames();
23     expect(names.length == 4, true);
24   });
25 }
26
```



## Adding unit tests for ForecastProvider

```
✓ weather_cli_2016 768ms
✓ test\forecast_provider_test.dart 2/2 passe...
  ✓ test forecast length 611ms
  ✓ all forecasts start at the same UTC time...
> test\location_provider_test.dart 2/2 passe...
```

```
Run | Debug
test('test forecast length', () async {
  var locationProvider = LocationProvider();
  var forecastProvider = ForecastProvider(locationProvider);

  var now = DateTime.now().toUtc();
  var currentHour = now.hour;

  bool succeeded = await forecastProvider.fetchHourlyForecast();
  expect(succeeded, true);
  expect(forecastProvider.hourlyForecast.length, 24 - currentHour - 1);
});
```

```
Run | Debug
test('all forecasts start at the same UTC time', () async {
  var locationProvider = LocationProvider();
  var forecastProvider = ForecastProvider(locationProvider);

  bool succeeded = locationProvider.selectLocation("Sydney");
  expect(succeeded, true);
  succeeded = await forecastProvider.fetchHourlyForecast();
  expect(succeeded, true);
  var sydneyLocalTime = forecastProvider.hourlyForecast[0].localTime;

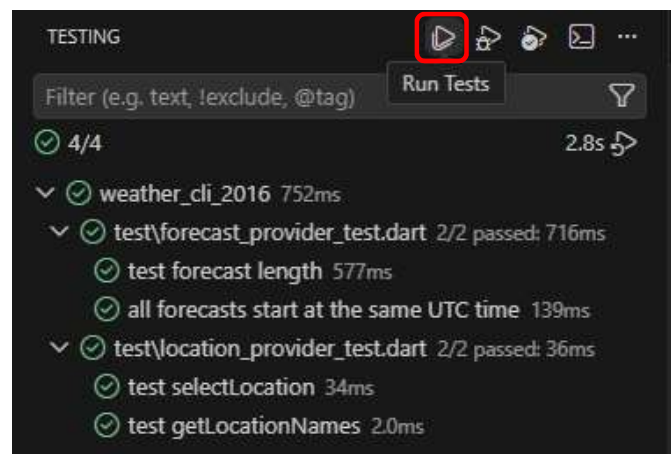
  succeeded = locationProvider.selectLocation("Aschaffenburg");
  expect(succeeded, true);
  succeeded = await forecastProvider.fetchHourlyForecast();
  expect(succeeded, true);
  var aschaffenburgLocalTime = forecastProvider.hourlyForecast[0].localTime;

  expect(
    sydneyLocalTime.microsecondsSinceEpoch ==
    aschaffenburgLocalTime.microsecondsSinceEpoch,
    true,
  );
});
```



# Running the unit tests

In VS Code



In Terminal

```
PS C:\flutter\code\weather_cli_2016> dart run test
00:00 +0: test\location_provider_test.dart: test selectLocation
selectLocation: location xxx not found
00:01 +4: All tests passed!
```



## Coverage of unit tests

TESTING

Filter (e.g. text, !exclude, @tag) Run Tests with Coverage

✓ 4/4 2.8s

- ✓ weather\_cli\_2016 752ms
- ✓ test\forecast\_provider\_test.dart 2/2 passed: 716ms
  - ✓ test forecast length 577ms
  - ✓ all forecasts start at the same UTC time 139ms
- ✓ test\location\_provider\_test.dart 2/2 passed: 36ms
  - ✓ test selectLocation 34ms
  - ✓ test getLocationNames 2.0ms

TEST COVERAGE

|                        |         |
|------------------------|---------|
| lib                    | 95.83%  |
| forecast_provider.dart | 94.44%  |
| location_data.dart     | 100.00% |
| location_provider.dart | 100.00% |
| weather_data.dart      | 100.00% |

```
19
20 Future<bool> fetchHourlyForecast() async {
21   if (!_initialized) {
22     _initialize();
23     _initialized = true;
24   }
25
26   hourlyForecast.clear();
27   var lat = _locationProvider.selectedLocation.latitude;
28   var lon = _locationProvider.selectedLocation.longitude;
29
30   // 'https://api.open-meteo.com/v1/forecast?latitude=49.97606&longitude=9.14163&h
31   final url =
32     "https://api.open-meteo.com/v1/forecast?latitude=$lat&longitude=$lon&hourly="
33
34   final response = await http.get(Uri.parse(url));
35   if (response.statusCode != 200) {
36     print(
37       "statusCode in response from open-meteo api was ${response.statusCode}",
38     );
39     return (false);
40   }
41 }
```





## Reaching 100% coverage for ForecastProvider

```
class LocationProvider {  
  final List<LocationData> _locations = [  
    LocationData(name: "Aschaffenburg", latitude: 49.97606, longitude: 9.14163),  
    LocationData(name: "Sydney", latitude: -33.86785, longitude: 151.20732),  
    LocationData(name: "New York", latitude: 40.71427, longitude: -74.00597),  
    LocationData(name: "Tokio", latitude: 35.6895, longitude: 139.69171),  
    // latitude must be between -90 (south) and +90 (north)  
    LocationData(name: "invalid", latitude: 90.1, longitude: 139.69171),  
  ];  
}
```

```
Run | Debug  
test('enforce API error', () async {  
  var locationProvider = LocationProvider();  
  var forecastProvider = ForecastProvider(locationProvider);  
  
  bool succeeded = locationProvider.selectLocation("invalid");  
  expect(succeeded, true);  
  succeeded = await forecastProvider.fetchHourlyForecast();  
  expect(succeeded, false);  
  expect(forecastProvider.hourlyForecast.length, 0);  
});
```

| TEST COVERAGE          |         |             |
|------------------------|---------|-------------|
| lib                    | 100.00% | <div></div> |
| forecast_provider.dart | 100.00% | <div></div> |
| location_data.dart     | 100.00% | <div></div> |
| location_provider.dart | 100.00% | <div></div> |
| weather_data.dart      | 100.00% | <div></div> |

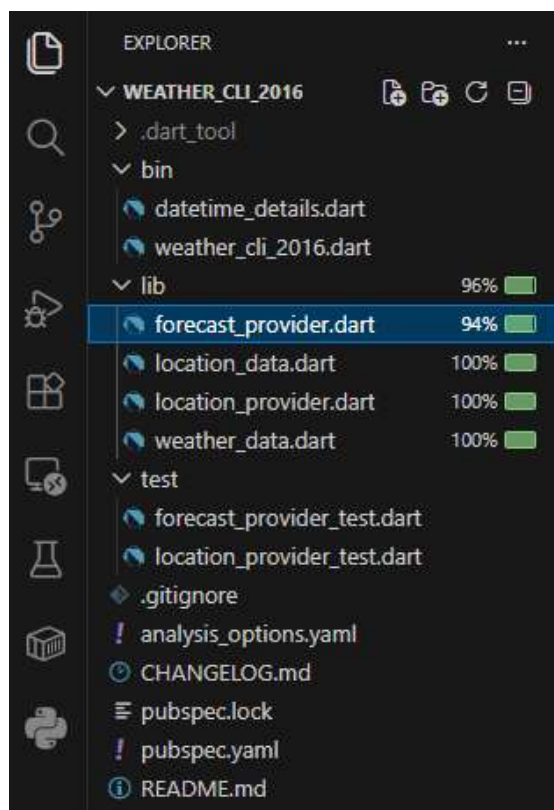
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TEST RESULTS  TERMINAL  
  
selectLocation: location xxx not found  
✓ test selectLocation  
✓ test getLocationNames  
✓ test forecast length  
✓ all forecasts start at the same UTC time  
statusCode in response from open-meteo api was 400  
✓ enforce API error  
  
Exited.
```



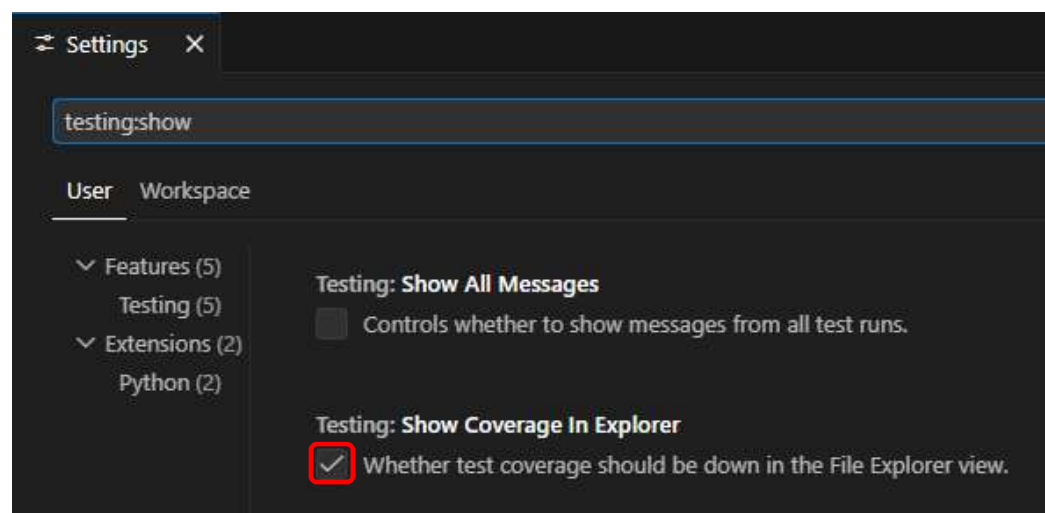


## Do not show coverage info in Explorer View

Once you ran unit tests with coverage in your VS Code session, the coverage results are also shown in Explorer View:



You can switch this off in the VS Code Settings:



Coverage results disappear when you close and re-open VS Code.



# Unit Tests for package fl\_chart ([https://pub.dev/packages/fl\\_chart](https://pub.dev/packages/fl_chart))

## Contributing

📦 Pull requests are welcome!

Remember that open-source projects thrive on contributions. Every improvement—no matter how small—makes a meaningful difference, even if it's just a single line.

Our documentation may contain grammatical issues, and if you're fluent in English, your help in correcting them would be greatly appreciated.

Check out [CONTRIBUTING.md](#), which contains a guide for those who want to contribute to the FL Chart.

Reporting bugs and issues is also a contribution, yes, it is.

## Test coverage (unit tests)

We should write unit tests for our written code. If you are not familiar with unit-tests, please start from [here](#).

[Mockito](#) is the library that we use to mock our classes. Please read more about it in their docs [here](#).

Our code coverage is calculated by [Codecov](#) (Our coverage is  at the moment)

When you push something in your PR (after approving your PR by one of us), you see a coverage report which describes how much coverage is increased or decreased by your code (You can check the details to see which part of your code made the change).

Please make sure that your code is **not decreasing** the coverage.



# KI can generate unit tests

<https://gemini.google.com/share/abedfb4d6a3f>

Generate unit tests for the following 2 classes in Dart: ▾

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:lat_lng_to_timezone/...
```



Writing unit tests for these classes is a great move, especially since `ForecastProvider` has several external dependencies (HTTP calls and timezone data) that can be tricky to handle in a testing environment.

To test these properly, we'll use the `test` and `mockito` (or `mocktail`) packages. Since `ForecastProvider` is tightly coupled with `http.get`, I recommend refactoring it slightly to accept an `http.Client` to make it mockable. However, for these tests, I will provide a solution using a Mock Client.

## Prerequisites

Add these to your `pubspec.yaml`:

YAML

```
dev_dependencies:
  test: ^1.24.0
  mockito: ^5.4.0
  http: ^1.1.0
```

Mockito is not really needed, http and test are already in pubspec.yaml.

## 1. Tests for LocationProvider

This is a "pure" Dart class, so it's straightforward to test.

Dart

```
import 'package:test/test.dart';
import 'package:your_project/location_provider.dart';

void main() {
  group('LocationProvider Tests', () {
    late LocationProvider provider;

    setUp(() {
      provider = LocationProvider();
    });

    test('Initial selected location should be Aschaffenburg', () {
      expect(provider.selectedLocation.name, equals('Aschaffenburg'));
    });

    test('getLocationNames returns all defined names', () {
      final names = provider.getLocationNames();
      expect(names, contains('Sydney'));
      expect(names, contains('New York'));
      expect(names.length, equals(5));
    });

    test('selectLocation updates the selected location if name exists', () {
      final result = provider.selectLocation('Sydney');
      expect(result, isTrue);
      expect(provider.selectedLocation.name, equals('Sydney'));
    });

    test('selectLocation returns false and does not change selection if name mis', () {
      final result = provider.selectLocation('Mars');
      expect(result, isFalse);
      expect(provider.selectedLocation.name, equals('Aschaffenburg'));
    });
  });
}
```



# KI can generate unit tests

<https://gemini.google.com/share/abedfb4d6a3f>

Would you like me to show you how to refactor the `ForecastProvider` to make it fully testable with a Mock Client?

To make your code testable and robust, we need to apply **Dependency Injection**. By passing the `http.Client` into the constructor, we can swap the real network client for a "Mock" client during tests. This prevents your tests from failing due to poor internet or API rate limits.

## 1. The Refactored ForecastProvider

I've updated the constructor to accept an optional `http.Client`. If none is provided, it defaults to the standard one.

```
Dart

class ForecastProvider {
  ForecastProvider(this._locationProvider, {http.Client? client})
    : _client = client ?? http.Client();

  final LocationProvider _locationProvider;
  final http.Client _client; // Use this instead of the static http.get
  List<WeatherData> hourlyForecast = [];
  bool _initialized = false;

  final url =
    "https://api.open-meteo.com/v1/forecast?latitude=$lat&longitude=$lon&hourly=temperature_2m,precipitation_probability";

  // Use the injected _client
  final response = await _client.get(Uri.parse(url));

  if (response.statusCode != 200) {
    return false;
  }
}
```

```
test('returns true and populates list on successful API call', () async {
  // 1. Create a Mock Client that returns a fake JSON response
  final mockClient = MockClient((request) async {
    final jsonResponse = {
      'hourly': {
        'time': ["2050-01-01T12:00", "2050-01-01T13:00"], // Future dates
        'temperature_2m': [25.0, 26.5],
        'precipitation': [0.0, 0.0],
        'cloud_cover': [0, 10]
      }
    };
    return http.Response(jsonEncode(jsonResponse), 200);
  });

  final forecastProvider = ForecastProvider(locationProvider, client: mockClient);

  // 2. Execute
  final result = await forecastProvider.fetchHourlyForecast();

  // 3. Verify
  expect(result, isTrue);
  expect(forecastProvider.hourlyForecast.length, equals(2));
  expect(forecastProvider.hourlyForecast[0].temp, 25.0);
});
```



# KI can generate unit tests

<https://gemini.google.com/share/abedfb4d6a3f>



```
test('returns false when API returns 404 or 500', () async {
  final mockClient = MockClient((request) async {
    return http.Response('Not Found', 404);
  });

  final forecastProvider = ForecastProvider(locationProvider, client: mockClient);
  final result = await forecastProvider.fetchHourlyForecast();

  expect(result, isFalse);
  expect(forecastProvider.hourlyForecast, isEmpty);
});
```

```
test('filters out past forecast times', () async {
  // This test ensures the "isAfter(nowInUtc)" logic works
  final mockClient = MockClient((request) async {
    final jsonResponse = {
      'hourly': {
        'time': ["2000-01-01T12:00", "2050-01-01T12:00"], // One past, one future
        'temperature_2m': [10.0, 30.0],
        'precipitation': [0.0, 0.0],
        'cloud_cover': [0, 0]
      }
    };
    return http.Response(jsonEncode(jsonResponse), 200);
  });

  final forecastProvider = ForecastProvider(locationProvider, client: mockClient);
  await forecastProvider.fetchHourlyForecast();

  // Should only contain the year 2050 entry
  expect(forecastProvider.hourlyForecast.length, equals(1));
  expect(forecastProvider.hourlyForecast[0].temp, 30.0);
});
```